

Product Engineer Test Assignment – Planner Application MVP

1. Assignment Overview

We are looking for a **Product Engineer** who can understand product requirements, define a practical MVP, and translate the requirements into a scalable technical solution.

For this assignment, you will work on the concept of a **Daily Planner / To-Do List application**.

The objective is to design and build the **first and smallest usable version of the application**.

Core MVP

The basic flow should be:

User selects a template → App downloads the template from the backend → App displays the template.

The application should be completely dependent on the backend for available templates. Templates should **not be hard-coded inside the mobile application**.

2. Product Objective

The application should allow users to:

- Browse available planner templates
- Select a template
- Preview the selected template
- Download the template from the server
- Create a planner using the selected template
- Edit the planner
- Add different types of content
- Set reminders
- Save the planner
- Access saved planners later
- Update or complete planner items

- Automatically save changes
 - View planner history/calendar
-

3. Reference Application

Please use the following application as the **primary reference** for understanding the product concept, user experience, and planner functionality.

Daily Planner / To-Do List – Google Play

[Reference Application – Daily Planner / To-Do List](#)

Please study the reference application to understand:

- Overall user experience
- Planner/template concept
- Navigation
- Template selection
- Planner creation
- Editing experience
- Core functionality

You are **not expected to copy the application exactly**. Use it as a reference to understand the expected product direction and user experience.

4. Main User Flow

The expected user flow is:

Open App

↓

Home

↓

Template Library



Select Template



Template Preview



Download Template from Server



Create Planner Page



Edit Planner



Set Reminder



Save



My Planners



Reminder Notification



Open Planner



Update / Complete



Auto-save



History / Calendar

5. Planner Editing Features

The planner editing screen should support the following capabilities:

Content

- Add Text / Notes
- Add Tasks
- Add Checklist
- Add Time / Schedule
- Add Habit
- Add Image
- Add Sticker

Editing

- Draw
 - Change Font
 - Change Color
 - Resize elements
 - Move elements
 - Delete elements
 - Duplicate elements
-

6. Additional Planner Features

The planner should also support:

- Adding notes
- Adding tasks
- Reminders
- Font family changes
- Bullets
- Numbered lists

- Undo
 - Redo
 - Save to gallery
 - Auto-save
 - History
 - Calendar view
-

7. Candidate Task

Based on the above requirements, explain and demonstrate **how you would approach building this MVP**.

Your solution should focus on the **smallest practical MVP** rather than trying to build the complete application.

Please explain:

A. Product Understanding

- What do you understand from the requirements?
- Who is the target user?
- What is the primary user problem?
- What would you consider the core MVP?
- Which features would you prioritize first and why?

B. User Experience

Create a simple user flow/wireframe showing how a user would:

1. Open the application
2. Browse templates
3. Select a template
4. Preview the template
5. Download the template
6. Create a planner
7. Edit the planner
8. Save the planner
9. Access the planner again

C. Technical Architecture

Explain how you would technically build the application.

Include:

- Mobile application architecture
- Backend architecture
- API structure
- Database structure
- Template storage
- Planner data storage
- Authentication, if required
- File/image storage
- Notification/reminder system
- Auto-save mechanism

D. Backend-Dependent Templates

A key requirement is that templates must come from the backend and **must not be hard-coded inside the mobile application**.

Explain:

- How templates would be stored
- How the mobile app would retrieve templates
- What API you would create
- How template metadata would be managed
- How new templates could be added without releasing a new version of the mobile app
- How you would handle template download failures

E. Database Design

Provide a basic database/schema design for:

- Users
- Templates
- Planners
- Planner elements/content
- Reminders
- Planner history

You may provide this as an ER diagram, table structure, or JSON structure.

F. API Design

Provide sample APIs required for the MVP.

For example:

- Get templates
- Get template details
- Download template
- Create planner
- Update planner
- Save planner
- Get user planners
- Delete planner
- Create reminder

For each API, briefly explain the purpose and expected request/response.

G. Error Handling

Explain how your application would handle situations such as:

- No internet connection
- Template download failure
- Server/API failure
- Auto-save failure
- Invalid template
- Large images/files
- User closing the application before saving
- Notification/reminder failure

H. Performance & Scalability

Explain how you would ensure:

- Fast template loading
 - Efficient image/file loading
 - Minimal API calls
 - Reliable auto-save
 - Smooth editing experience
 - Scalability as the number of users and templates increases
-

8. MVP Prioritization

Divide the features into:

Must Have

Features required for the first working MVP.

Should Have

Features that improve the product but can be added after the initial MVP.

Nice to Have

Features that are not required for the first version.

Please explain the reasoning behind your prioritization.

9. Technical Requirements

Mobile Application

- Build the mobile application as a **Native Android application**.
- Use **Kotlin** as the programming language.
- **No iOS application is required** at this time.

Backend

You do **not** need to build the backend from scratch.

Use **Supabase** or a **suitable alternative** for backend services such as:

- Database
- Authentication, if required
- Storage
- Backend/API services

If you choose an alternative to Supabase, please explain why.

10. AI Tools

We expect you to use **AI tools efficiently and effectively** during the development process.

You may use AI tools for:

- Code generation and assistance
- Debugging
- Research
- Technical problem-solving
- Documentation
- Testing
- Improving development efficiency

The use of AI is encouraged, but the implementation should be **scalable, maintainable, and well understood by you**.

You should be able to explain the code, architecture, and technical decisions made with the help of AI tools.

11. Technical Stack

Recommend and use the appropriate technology stack for the assignment.

At minimum, consider:

- Native Android / Kotlin
- Backend
- Database
- Cloud/storage
- Authentication, if required
- Notification service
- Hosting/deployment
- Any required third-party services

Explain **why** you selected the technologies.

12. Deliverables

Please submit:

1. **Product understanding**
2. **User flow**
3. **Basic UI/wireframes**
4. **Technical architecture**
5. **Database/schema design**
6. **API design**
7. **MVP feature prioritization**
8. **Technical stack recommendation**
9. **Error/edge-case handling**
10. **Working Android application/prototype**
11. **Source code**

The implementation does not need to be fully production-ready, but the solution should demonstrate good engineering practices and a clear understanding of the requirements.

13. Evaluation Criteria

The assignment will be evaluated based on:

Product Thinking – 20%

- Understanding of the user problem
- Understanding of requirements
- MVP thinking
- Feature prioritization

Technical Understanding – 25%

- Android/Kotlin implementation
- Architecture
- Backend/API understanding
- Database design
- Data flow

Problem Solving – 20%

- Edge cases
- Error handling
- Practical solutions
- Trade-off analysis

UX & User Flow – 15%

- Clear user journey
- Simple and practical UX
- Ease of use

Scalability & Performance – 10%

- Performance considerations
- Scalability
- Efficient data handling

Communication – 10%

- Clarity of explanation
 - Documentation quality
 - Ability to explain technical decisions
-

14. Expected Approach

We are **not expecting a fully production-ready application**.

The goal of this assignment is to understand how you:

- Analyze a product requirement
- Break a problem into smaller components
- Define an MVP
- Make technical decisions
- Build a Native Android application using Kotlin
- Work with a backend service such as Supabase
- Design APIs and data structures
- Use AI tools effectively during development
- Think about scalability and edge cases
- Balance product requirements with engineering effort

Focus on **clarity, practicality, scalability, and good product/engineering decisions**.

15. Development Timeline & Progress Updates

You should complete the assigned task **within 2 days of receiving the assignment**.

If you need additional time, please communicate this in advance. Additional time may be provided based on the situation.

During the assignment, please **share a brief progress update every day**, including:

- What you completed
 - What you are currently working on
 - Any blockers or challenges
 - What you plan to complete next
-

16. Submission Guidelines

Please keep your submission **concise and focused on the MVP**.

Clearly mention:

- Your assumptions
- Your technical decisions
- Your reasoning
- Features included in the MVP
- Features intentionally excluded from the MVP
- Any questions or ambiguities you identified

If you choose a different approach from the requirements, explain **why**.

Submission Deadline

Once completed, please send your final submission to:

hello@lumoslogic.com

Please include the following in your submission:

- APK / working application
 - Source code repository
 - Documentation
 - UI/wireframes
 - Architecture/database/API details
-